- *Pull the authorization out into a filter*. As you saw in chapters 5 and 7, endpoint filters can be useful for extracting common cross-cutting concerns, such as validation and authorization. You may find that endpoint filters help reduce the duplication in your endpoint handlers, though this often comes at the expense of additional complexity in the filter itself, as well as a layer of indirection in your handlers. You can see this approach in the source code accompanying this chapter.
- *Push the authorization responsibilities down into the domain.* Instead of performing the resource-based authorization in your endpoint handlers, you could run the checks inside the domain instead, in the `RecipeService` in this case. This has advantages, in that it often reduces duplication, keeps your endpoints simpler, and ensures that authorization checks are always applied regardless of how you call the domain methods.

  The downside to this approach is that it may cause your domain/application model to depend directly on ASP.NET Core-specific constructs such as `IAuthorizationService`. You can work around this by creating a wrapper façade around the `IAuthorizationService`, but this may also add some complexity. Even if you take this approach, you typically want to apply declarative authorization policies to your endpoints as well to ensure that the endpoint executes only for users who could possibly be authorized.

There's no single best answer on which approach to take; it will vary depending on what works best for your application. Authentication and authorization are inevitably tricky subjects, so it's important to consider them early and design your application with security in mind.